



Programação Avançada

Ano Letivo 2024 – 2025

Trabalho Prático: Jogo de Xadrez

2022107363 - Davi Nasser Torres Gama

2021104069 - Miguel Francisco Pires Lopes

2020152127 - Ruben Apolinário Almeida

1 Índice

1	Índice	2
2	Introdução	3
3	Decisões de Implementação	4
4	Padrões de Programação Aplicados.....	6
4.1	MVC (<i>Model-View-Controller</i>)	6
4.2	Memento	8
4.3	Factory	10
4.4	Facade	12
4.5	Observable e Observer	13
5	Descrição de Classes	15
5.1	Classes do modelo (pt.isec.pa.chess.model).....	15
5.2	Classes do modelo de dados (pt.isec.pa.chess.model.data)	16
5.3	Classes da UI (pt.isec.pa.chess.ui).....	19
5.4	Classes de recursos da UI (pt.isec.pa.chess.ui.res).....	21
6	Relacionamentos Entre Classes	22
6.1	Diagrama UML Geral	22
7	Funcionalidades e Estado de Implementação	24
8	Conclusão	29
9	Referências.....	30

2 Introdução

O desenvolvimento desta aplicação de Xadrez em Java surge da necessidade de consolidar diversos conceitos avançados de programação, tais como a adoção de padrões de projeto, a separação de camadas pelo modelo MVC (Model-View-Controller) e a edição dinâmica de tabuleiro. Baseada em JavaFX para a interface gráfica, a solução integra recursos multimedia (imagens e sons), persistência de estado (serialização e ficheiros de texto), funcionalidades avançadas do jogo (roque, en passant, promoção de peão, validação de xeque e xeque-mate) e mecanismos de undo/redo através do padrão Memento.

Ao longo deste trabalho, apresentamos o percurso de implementação: desde a definição das classes do modelo (representando peças, tabuleiro, jogador e regras), passando pela fachada ChessGameManager (que faz a ponte entre UI e lógica de jogo), até à construção dos componentes visuais (como BoardCanvas, MenuUI e StatusBarPane) que fornecem uma experiência intuitiva e responsiva ao utilizador. Além disso, implementámos um modo de edição livre do tabuleiro e um modo de aprendizagem que destaca movimentos válidos, oferecendo uma plataforma robusta para estudo e prática do jogo.

O relatório acompanha todas as decisões técnicas tomadas (como a escolha de fábricas para criação de peças e a implementação de listas de observadores para notificar a UI), os padrões aplicados (MVC, Memento, Factory e Observer) e a forma como cada classe contribui para o conjunto. Por fim, documentamos o cumprimento de cada funcionalidade prevista e sinalizando pontos já concluídos, de modo a assegurar uma visão completa do projeto e facilitar a sua continuidade.

3 Decisões de Implementação

Optámos por organizar o sistema segundo o padrão MVC para manter clara a separação entre lógica de jogo, interface gráfica e controlo de fluxo. O modelo foi estruturado em várias camadas: as classes de domínio representam as entidades básicas (peças e tabuleiro), apoiadas por fábricas (PieceFactory e PieceFactoryText) que abstraem a criação de objetos a partir de enums ou texto. Para garantir que a UI nunca acesse diretamente o estado interno do modelo, toda a comunicação passa pela fachada ChessGameManager, que expõe métodos claros como movePiece ou getValidMoves.

Na camada de persistência, decidimos suportar o formato binário, via ChessGameSerialization, para guardar/recarregar partidas completas (.dat) e de texto, por meio de métodos de importação/exportação em ChessGame, permitindo editar posições manuais ou compartilhar configurações simples. Esse duplo formato segue as recomendações das aulas e torna o sistema flexível.

Para implementar undo/redo mantivemos o padrão Memento: antes de cada movimento, o CareTaker armazena um clone completo de GameBoard (e, consequentemente, do estado de cada Pawn, incluindo pawnStatus). Isso elimina a necessidade de lógica complexa de “desfazer” dentro das classes de peça. Ao restaurar um memento, o tabuleiro volta exatamente ao ponto anterior, preservando vulnerabilidades ao en passant ou roque até o instante correto.

Optámos por delegar todo o tratamento do en passant à própria classe Pawn (via checkPawnMove), em vez de espalhar essa regra no GameBoard. Dessa forma, cada instância de Pawn avalia se um movimento diagonal vazio caracteriza captura en passant, consultando apenas seu próprio pawnStatus. Para que funcione apenas no lance seguinte, GameBoard.movePiece limpa o pawnStatus de todos os peões antes de executar qualquer movimento, e o método updatePosition do Pawn atualiza adequadamente o estado para ONCE ou MORETHANONCE.

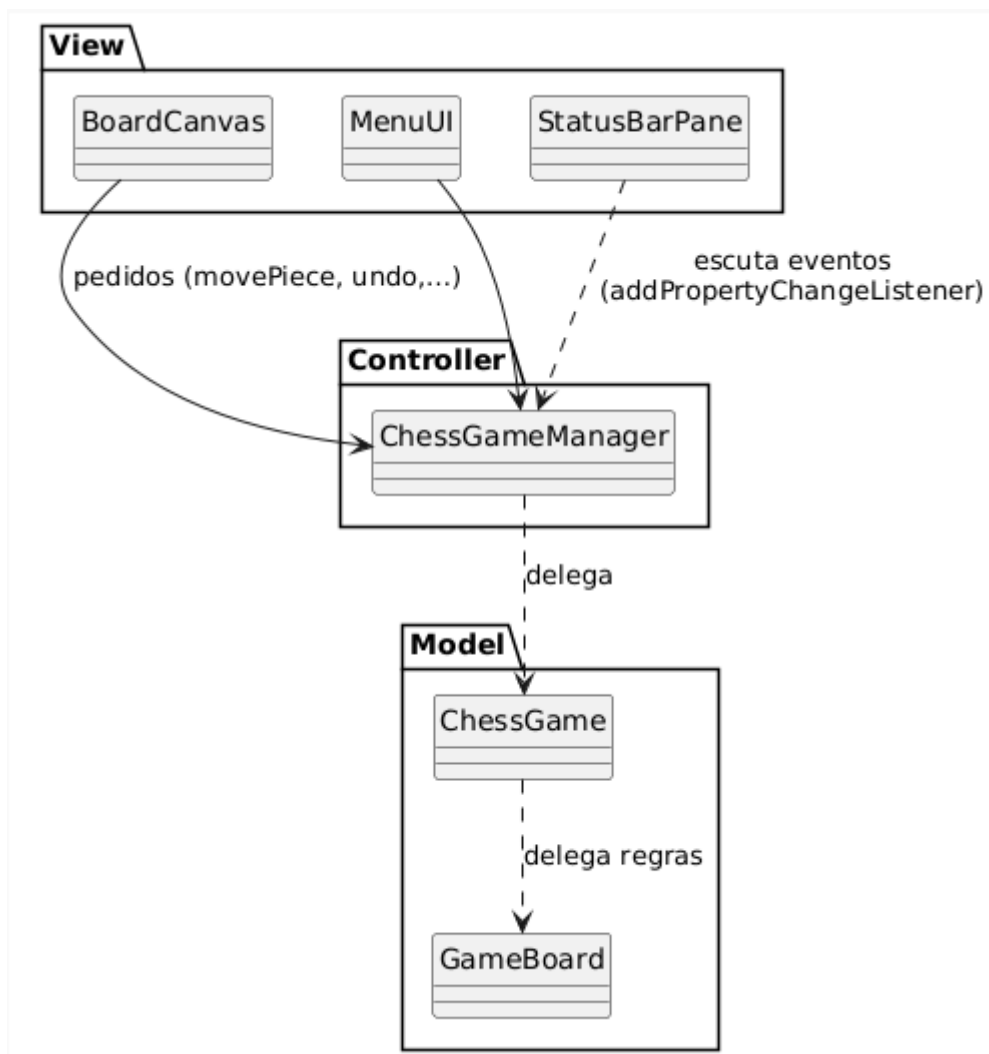
No front-end, recorremos a JavaFX por oferecer facilidade na construção de interfaces ricas. O MainJFX lança a aplicação e carrega o RootPane, que organiza o ecrã numa BorderPane: a zona superior hospeda o menu (MenuUI), o centro mostra IntroPane ou BoardCanvas, e a parte inferior agrupa painéis de informação do jogador e a barra de estado (StatusBarPane). O BoardCanvas desenha o tabuleiro em canvas, carregando imagens via ImageManager e sons via SoundManager. Cada clique no tabuleiro é transformado em coordenadas de coluna/linha para chamar ChessGameManager.movePiece ou, em “Learning Mode”, getValidMoves para destacar possibilidades.

Por fim, optámos por centralizar a gestão de recursos multimédia (ImageManager e SoundManager) para nunca recarregar imagens ou sons diversas vezes e aplicar um único ponto de controlo para tocar efeitos. Na escolha de alertas e diálogos, adotamos um singleton AlertManager que uniformiza mensagens de confirmação ou informação, garantindo consistência visual. Estas decisões, como a adoção do padrão MVC, o recurso a Memento para desfazer/refazer, fábricas para criação de peças, a delegação do en passant à classe Pawn e a utilização do JavaFX em camadas bem definidas, permitem um código modular, simples de testar e manter, além de garantir que cada componente cumpra a sua responsabilidade sem misturar a lógica de negócio com a apresentação.

4 Padrões de Programação Aplicados

Nesta secção descrevemos os padrões de projeto utilizados com base no conteúdo ministrado pelo professor durante as aulas, explicando como cada padrão foi aplicado à estrutura do nosso jogo de xadrez em Java.

4.1 MVC (Model-View-Controller)



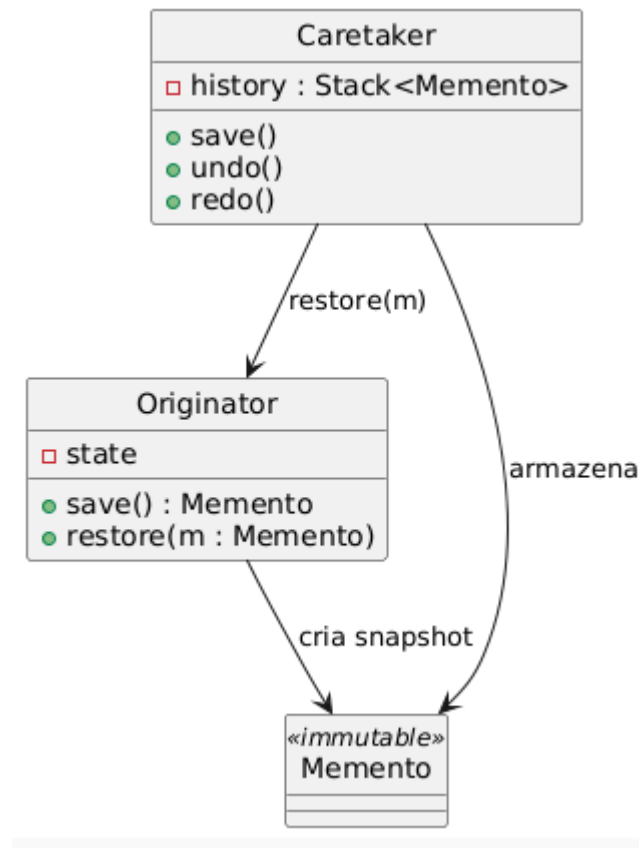
IMPLEMENTAÇÃO

- **Model (Modelo):** contém toda a lógica de jogo e as estruturas de dados. Aqui agrupamos classes como `ChessGame`, `GameBoard` e as subclasses de `Piece` (`Pawn`, `Rook`, `Knight`, etc.). Essas classes representam o estado completo do tabuleiro, as regras de movimento, detecção de xeque/xeque-mate, en passant, promoção de peão e remoções de peças.

- View (Visão): reúne todos os componentes de interface que exibem informações ao utilizador e capturam interações gráficas, mas não contêm lógica do modelo. Exemplo: BoardCanvas (responsável por desenhar tabuleiro e peças), MenuUI (bar-ra de menus), StatusBarPane, PlayerInfoPane, IntroPane e LogWindow. Esses componentes apenas chamam métodos de controlo e respondem a eventos enviados pelo controller.
- Controller (Controlador): corresponde à classe ChessGameManager, que recebe comandos vindos da View (por exemplo, cliques no tabuleiro, opções de menu), executa a lógica no Modelo, atualiza o estado de cada peça, valida movimentos e, em seguida, dispara notificações para que a View seja atualizada. No fluxo de execução, sempre que o utilizador tenta mover uma peça, o BoardCanvas invoca ChessGameManager.movePiece(...). O controller, por sua vez, chama GameBoard.movePiece(...), valida se há xeque, alterna o turno e, por fim, informa a View para redesenhar tudo.

Essa clara separação permite que façamos alterações na interface gráfica (View) sem tocar nas regras do jogo (Model), e vice-versa. O ChessGameManager faz a mediação e garante que cada camada cumpra apenas a sua responsabilidade, conforme indicado pelo professor em aula.

4.2 Memento



IMPLEMENTAÇÃO

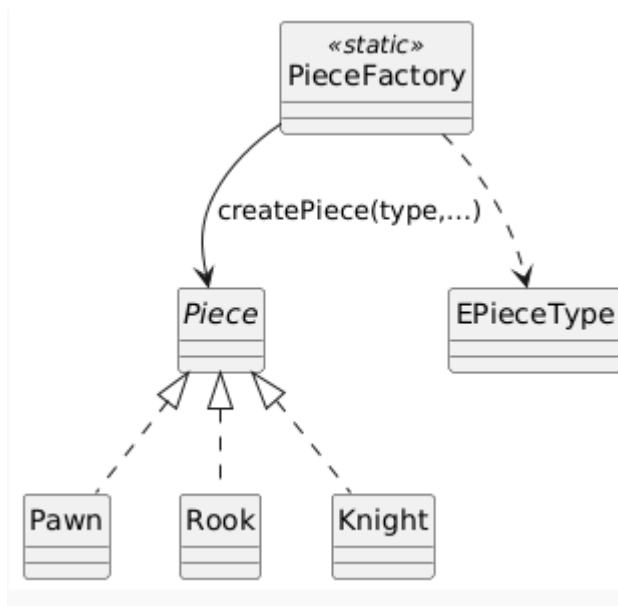
O padrão Memento foi aplicado para permitir desfazer e refazer jogadas (undo/redo) sem expor detalhes internos do tabuleiro ou das peças. Seguindo a explicação do professor:

- **Originator**: representa o objeto cujo estado deve ser salvo. No nosso caso, a responsabilidade cabe à classe `ChessGame` (que incorpora internamente o `GameBoard`). Quando solicitamos “guardar” antes de um movimento, o método `ChessGame.save()` cria um memento que encapsula todo o estado atual do tabuleiro, incluindo a posição de cada peça e seus atributos internos (como `pawnStatus`).
- **Memento**: objeto imutável que armazena o estado interno do originator num determinado instante. Concretizamos isso por meio de uma classe interna (ou anônima) que implementa `IMemento`, garantindo que ninguém externo modifique o conteúdo do memento.

- Caretaker: a classe CareTaker mantém duas pilhas de mementos – uma para undo e outra para redo. Antes de cada movimento de peça, invocamos `careTaker.save()`, que chama `originator.save()` e armazena o memento no topo da pilha. Para desfazer, o CareTaker chama `originator.restore(...)` com o memento anterior, revertendo tudo exatamente ao estado salvo. Após um undo, é possível refazer executando o memento da pilha de redo.

Dessa forma, conseguimos implementar o mecanismo de undo/redo sem que as classes de peça ou o próprio GameBoard saibam de como o histórico está a ser guardado. Todo o trabalho de guardar e restaurar cabe ao CareTaker e ao contrato de IMemento e IOriginator, conforme demonstrado em aula.

4.3 Factory



IMPLEMENTAÇÃO

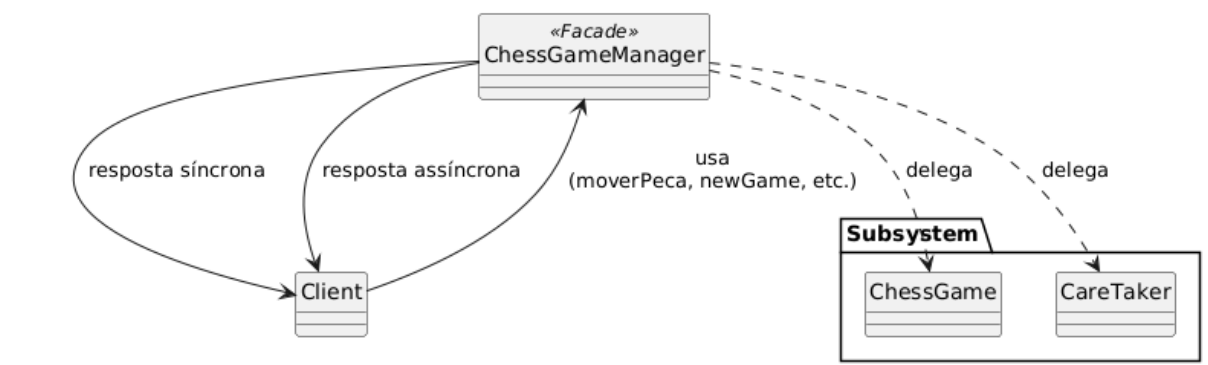
Para simplificar e centralizar a criação de objetos de peça, aplicámos o padrão Factory conforme explicado em aula:

- **PieceFactory**: recebe como parâmetros um **EPieceType**, um boolean que indica se é peça branca ou preta, e as coordenadas iniciais (coluna e linha). Internamente, executa um switch para instanciar a subclasse correta de **Piece** (por exemplo, `new Pawn(...)`, `new Rook(...)`, `new Knight(...)` etc.). O utilizador do factory (o `GameBoard.initializePieces()` e o `GameBoard.clone()`) nunca precisa conhecer detalhes de construtores específicos: basta indicar qual o tipo e a peça sai pronta para usar.
- **PieceFactoryText**: concebido para criar peças a partir de representação textual (por exemplo, no formato “pP e4” ou “rR a1”). Ao importar um ficheiro de texto, o `ChessGame.importGameState(...)` delega para **PieceFactoryText**, que faz parsing da string, extrai o tipo, a cor e a posição e invoca o **PieceFactory** para gerar a instância apropriada. Isso segue a recomendação do professor de isolar toda a lógica de conversão de texto em classes especializadas, mantendo o código de importação limpo e legível.

Com esse padrão, quando precisarmos de adicionar um novo tipo de peça ou alterar o processo de inicialização, bastará alterar a classe de factory, sem modificar o restante do modelo. A factory garante que a criação de objetos seja flexível, respeitando o

Princípio Aberto/Fechado e seguindo o exemplo do professor sobre como abstrair a construção de instâncias.

4.4 Facade



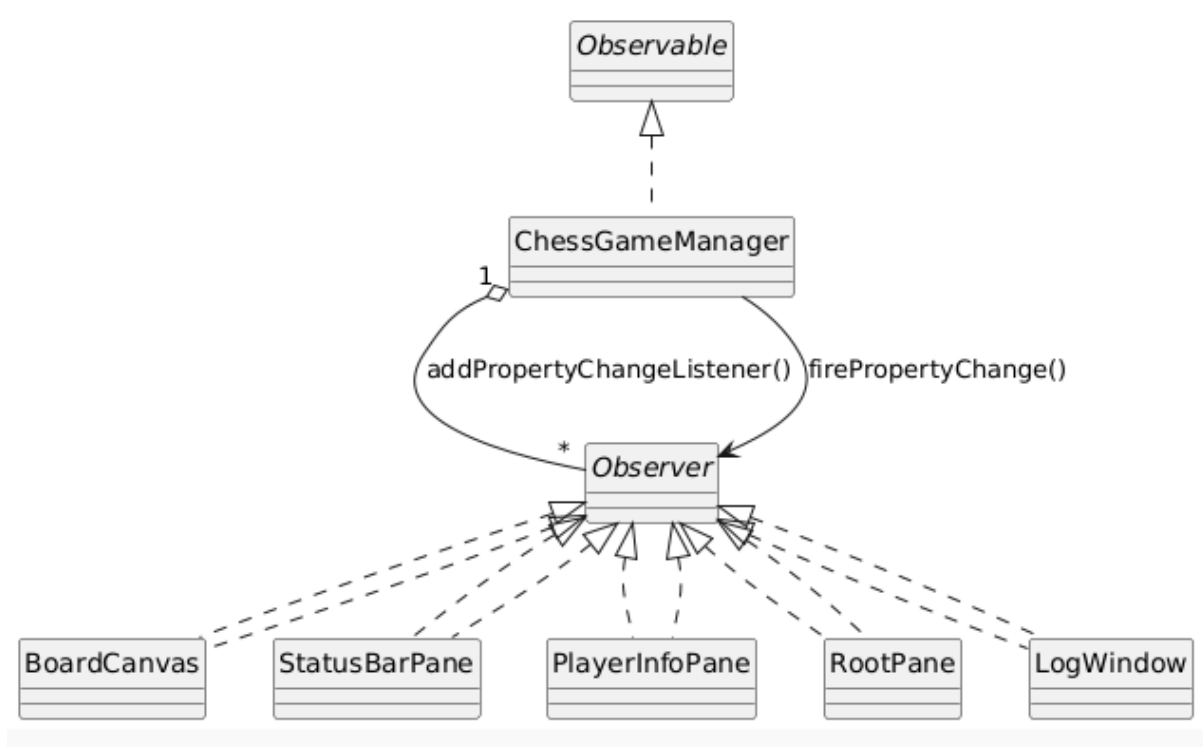
IMPLEMENTAÇÃO

A classe **ChessGameManager** funciona também como Facade. Toda a interface gráfica comunica exclusivamente com este objeto. Nenhuma View fala diretamente com **ChessGame**, **GameBoard** ou outras classes de lógica.

O **ChessGameManager** expõe métodos de alto nível: **newGame**, **movePiece**, **undo**, **redo**, **saveGame**, **loadGame**, **importGame**, **exportGame**. Escondendo pormenores como serialização, validação de jogadas, verificação de xeque-mate, gestão de histórico e deteção de empate.

Internamente, ele coordena **ChessGame** (regras), **CareTaker** (mementos) e **ModelLog** (registo de eventos). A View pede “move a peça de e2 para e4” e recebe “jogada válida / inválida”, sem precisar conhecer como o tabuleiro é clonado, se o rei fica em xeque ou como o histórico é armazenado. Isto segue o exemplo do professor de usar uma classe fachada para simplificar a utilização de sistemas complexos.

4.5 Observable e Observer



IMPLEMENTAÇÃO DO OBSERVABLE

No nosso projeto o papel de objeto observable é desempenhado pelo ChessGameManager. A classe mantém internamente uma instância de PropertyChangeSupport. Sempre que o estado do jogo muda chama firePropertyChange, publicando eventos como PROP_BOARD_CHANGED, PROP_TURN_CHANGED, PROP_CHECK, PROP_GAME_OVER e outros.

Todas as Views que precisam de saber quando algo acontece registam-se através de addPropertyChangeListener. Com isto a lógica de jogo permanece completamente independente de qualquer detalhe de interface, limitando-se a “emitir sinais” de que algo ocorreu, tal como foi exemplificado nas aulas.

IMPLEMENTAÇÃO DO OBSERVER

Os componentes da interface agem como observadores, BoardCanvas, StatusBarPane, PlayerInfoPane, RootPane e LogWindow chamam addPropertyChangeListener sobre o ChessGameManager e passam-lhe um callback que será executado sempre que o evento correspondente for disparado.

- Quando chega PROP_BOARD_CHANGED o BoardCanvas redesenha o tabuleiro.
- StatusBarPane reage a PROP_CHECK e exibe a mensagem “Rei em xeque”.

- RootPane recebe PROP_GAME_OVER e substitui o conteúdo central pelo IntroPane.

Deste modo cada janela actualiza-se sozinha, sem que o modelo tenha de saber quantas janelas existem ou como estão dispostas. Esta separação cumpre exactamente o padrão Observer mostrado pelo professor.

5 Descrição de Classes

5.1 Classes do modelo (pt.isec.pa.chess.model)

Inclui a classe que constitui a Facade de acesso à lógica, que internamente distribui as responsabilidades que lhe são pedidas:

Classe	Responsabilidade	Objetivo
ChessGameManager	Atuar como elo entre a UI e o modelo (Controller), receber comandos da interface, aplicar regras de negócio e notificar UI	Gerir o fluxo do jogo: delegar validação de movimentos ao ChessGame e GameBoard, controlar turnos, disponibilizar dados para a UI e propagar eventos (xeque, xeque-mate, undo/redo) para atualização da apresentação.
CareTaker	Gerir histórico de estados (Memento)	Armazena e recupera mementos para permitir operações de undo/redo no estado do jogo sem expor detalhes internos do modelo.
IMemento	Define métodos para o memento	Define a interface que qualquer memento deve implementar, encapsulando o estado do originator para restauração futura.
IOriginator	Define métodos para o originator	Declara métodos para salvar e restaurar estado em forma de memento, de modo que CareTaker possa manipular esses estados.

5.2 Classes do modelo de dados (pt.isec.pa.chess.model.data)

Contém as classes que representam as estruturas de dados e respetiva gestão:

Classe	Responsabilidade	Objetivo
ChessGame	Controlar o fluxo e o estado da partida	Agir como “orquestrador” das jogadas: invocar <code>GameBoard.movePiece</code> , validar xeque/xeque-mate, alternar turnos, gerir promoção de peão, importação/exportação de estado e notificar a UI.
ChessGameSerialization	Persistir e recuperar o estado completo do jogo em ficheiros (.dat)	Fornecer métodos para serializar (<code>saveGame</code>) e desserializar (<code>loadGame</code>) toda a instância de <code>ChessGame</code> , permitindo guardar e recarregar partidas.
ModelLog	Registrar eventos e ações internas do modelo	Gravar eventos como “movimento inválido” ou “xeque detectado” em memória, permitindo depuração e exibição no <code>LogWindow</code> .
IPlayable	Interface para objetos jogáveis	Especifica métodos que definem comportamento de entidades que podem “jogar” (por exemplo, iniciar, pausar, validar jogadas).
Player	Modelo de um jogador	Armazenar nome, equipa (cor das peças) e pontuação; usado por <code>ChessGameManager</code> para controlar turno e vitória.
PlayerData	Data transfer object para informações de jogador	Encapsular apenas as informações relevantes de <code>Player</code> (nome, equipa, pontuação) para envio seguro à UI.
GameBoard	Representar e gerir o tabuleiro de xadrez	Manter lista de peças, validar movimentos (incluindo roque, en passant e cheque), executar jogadas, promoção de peão e remoção de peças.
Piece	Superclasse de todas as peças do tabuleiro	Representa atributos e comportamentos genéricos de qualquer peça de xadrez (posição, cor, movimento, clonagem), servindo de base

		para subclasses específicas como Pawn, Rook, Knight etc.
Position	Representar coordenadas no tabuleiro	Definir colunas e linhas do tabuleiro (ex.: 'e4' → coluna 4, linha 3) para uso consistente em todas as peças e movimentos.
PositionData	Estrutura simples para retorno de dados de promoção de peão	Indicar a coluna e a linha onde um peão atingiu a última fileira, servindo como sinalizador para o UI abrir diálogo de promoção.
PieceFactory	Gerar instâncias de peças com base em enumerações	Criar objetos concretos de peças (Pawn, Rook, Knight, etc.) dado um valor de EPieceType, cor da equipa e posição inicial, abstraindo a lógica de construção das subclasses.
PieceFactoryText	Gerar instâncias de peças a partir de texto	Converter representações textuais (e.g. "pP e4") em objetos Piece correspondentes, usado principalmente na importação de jogos a partir de ficheiro de texto (.txt).
Pawn	Implementar regras e comportamentos específicos do peão	Definir deslocamentos (avanço de 1 ou 2 casas), capturas normais, en passant e promoção; atualizar estado interno (pawnStatus, wasMoved).
Rook	Implementar regras e comportamentos específicos da torre	Definir movimento em linha reta (horizontais e verticais), verificar caminho livre, capturar peças adversárias e participar no roque.
Knight	Implementar regras e comportamentos específicos do cavalo	Definir movimento em "L" (duas casas numa direção e uma na perpendicular), pular peças intermediárias, capturar peças adversárias e atualizar posição.
Bishop	Implementar regras e comportamentos específicos do bispo	Definir movimento livre em diagonais, verificar caminho livre e capturar peças adversárias.
King	Implementar regras e comportamentos específicos do rei	Definir movimento de uma casa em qualquer direção, validar roque (verificar se rei e torre nunca se moveram, caminho

		livre e sem xeque), capturar peças adversárias e atualizar posição.
Queen	Implementar regras e comportamentos específicos da dama	Definir movimento livre em todas as direções (linhas, colunas e diagonais), validar caminho livre e capturar peças adversárias.
EPawnMove	Enumerar estados de movimento dos peões	Definir os valores NEVER, ONCE, MORETHANONCE para representar se um peão nunca se moveu, moveu duas casas na última jogada ou já moveu anteriormente, subsidiando a regra de en passant.
EChessState	Representar estados possíveis de uma partida de xadrez	Indicar em que fase ou condição o jogo se encontra (por exemplo, NÃO_INICIADO, EM_PROGRESSO, XEQUE, XEQUE_MATE, EMPATE).
EPieceType	Enumerar os tipos de peça de xadrez disponíveis	Fornecer constantes para os diferentes tipos de peças (PAWN, ROOK, KNIGHT, BISHOP, QUEEN, KING), usadas na criação e identificação de objetos Piece.
ETeamColor	Identificar as cores/equipes de cada jogador	Diferenciar as equipes (BRANCA, PRETA) para controlar turnos, validar movimentos e determinar adversário ou condições de vitória.

5.3 Classes da UI (pt.isec.pa.chess.ui)

Classes que implementam a interface em modo gráfico em Java Fx:

Classe	Responsabilidade	Objetivo
ChessApp	Ponto de entrada principal da aplicação	Inicializa o ambiente JavaFX e configura o MainJFX como aplicação principal, carregando o stage.
MainJFX	Classe Application do JavaFX	Faz o launch da aplicação, cria o Stage principal, instancia o RootPane e chama show().
RootPane	Layout principal da aplicação (container BorderPane)	Agrupa e posiciona todos os componentes da UI: inclui o MenuUI no topo, o IntroPane ou BoardCanvas no centro e os PlayerInfoPane nas extremidades. Regista listeners para eventos do ChessGameManager (xeque, checkmate) e dispara alertas, além de controlar o redimensionamento e reposicionamento dos elementos ao mudar o tamanho da janela.
LogWindow	Janela de registo de eventos	Exibe mensagens de log e diagnósticos (movimentos inválidos, cheque, debug), servindo como consola auxiliar.
BoardCanvas	Canvas de desenho do tabuleiro de xadrez	Renderiza o tabuleiro, as peças e os destaques de seleção; captura cliques do utilizador para tentar movimentos.
IntroPane	Painel de boas-vindas	Apresenta a tela inicial (logo, título e botões) antes de iniciar qualquer partida ou carregar jogo.
MenuUI	Construção e gestão da barra de menu	Cria os menus “Game” e “Mode” (incluindo itens como New, Open, Save, Learning Mode, Undo/Redo) e associa cada item a handlers que invocam métodos do ChessGameManager para iniciar novo jogo, carregar, guardar, ativar/desativar

		modos, desfazer/refazer movimentos e sair.
PlayerInfoPane	Painel de visualização de informações do jogador	Exibe, em formato compacto, o nome, equipa (brancas/pretas) e pontuação do jogador atual, servindo como indicador de estado e contribuindo para a área de estatísticas na UI.
StatusBarPane	Barra de estado no rodapé	Exibe informações dinâmicas sobre o turno atual (nome do jogador e cor), estado do jogo (xeque, checkmate) e outras mensagens de status; atualiza-se em resposta a eventos vindos do ChessGameManager.
FileChooserService	Abstração do diálogo de seleção de ficheiros	Fornece métodos para abrir e guardar ficheiros sem expor diretamente APIs do JavaFX, simplificando operações de I/O na UI.
AlertManager	Gestão de caixas de diálogo e alertas	Exibe alertas informativos, de confirmação e caixas de entrada (dialogs), padronizando estilos e lógica de confirmação.
IAlertManager	Interface do AlertManager	Define o contrato para implementar caixas de alerta e confirmação, permitindo injeção de dependência e testes automatizados.

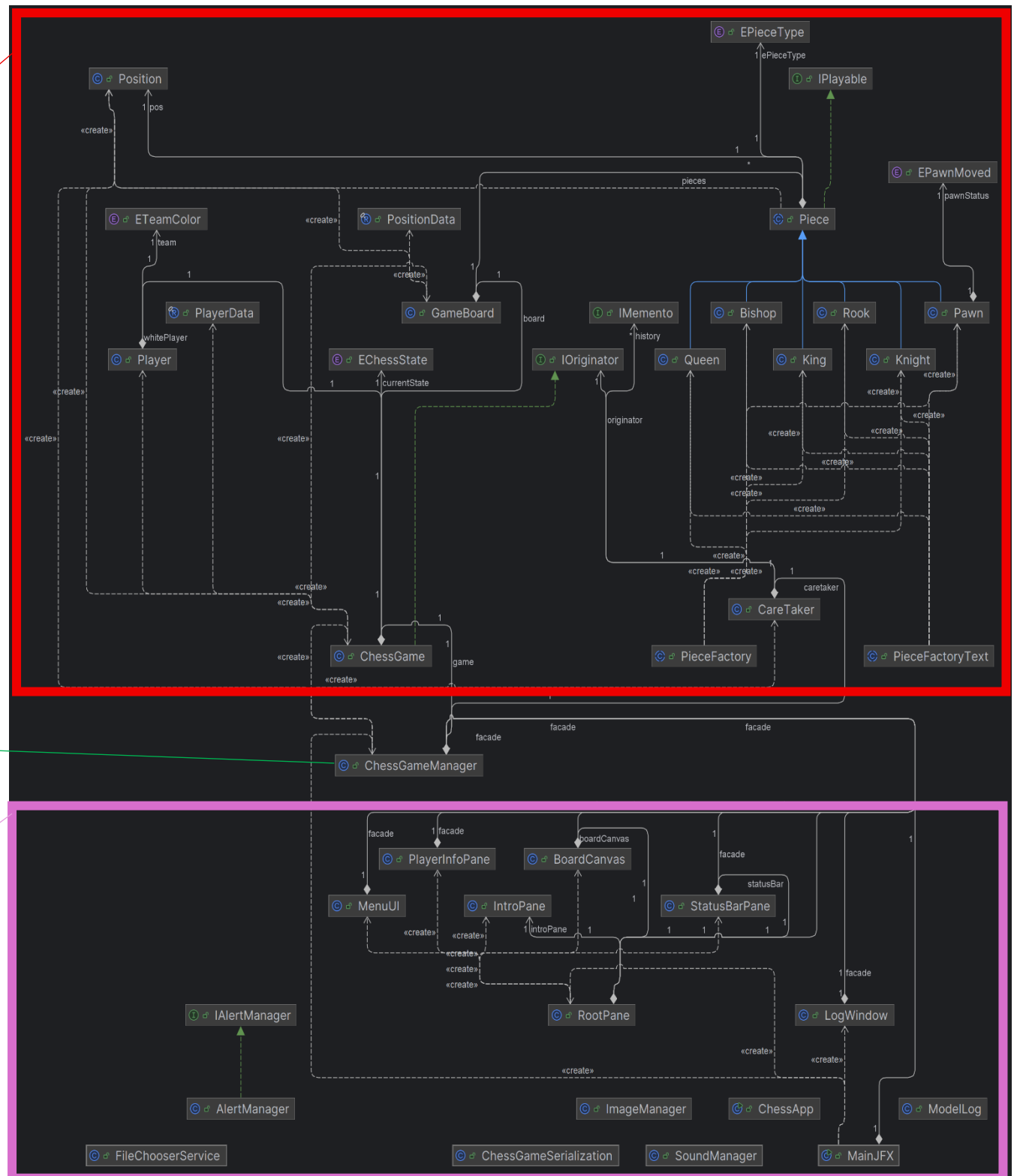
5.4 Classes de recursos da UI (pt.isec.pa.chess.ui.res)

Package base para gestão dos recursos multimédia da aplicação, recorrendo, por exemplo, ao padrão multiton:

ImageManager	Gestão centralizada de imagens	Carrega, armazena em cache e disponibiliza imagens (ícones de peças, fundos) para toda a UI, evitando recarregamentos repetidos.
SoundManager	Gestão de efeitos sonoros	Toca sons de movimento e eventos (captura, cheque, xeque-mate), fornecendo interface simples para sequências de áudio.

6 Relacionamentos Entre Classes

6.1 Diagrama UML Geral



VISÃO GERAL

O diagrama divide o projeto em três camadas. Na base está a interface gráfica, no centro o gestor de jogo (facade) e no topo o núcleo de dados.

CAMADA DE INTERFACE

RootPane age como palco principal e instancia IntroPane, BoardCanvas, PlayerInfoPane, StatusBarPane e MenuUI. Cada componente recebe uma referência do objeto ChessGameManager, o que lhes permite pedir informação ou executar ações sem conhecer os pormenores do modelo.

LogWindow também escuta o gestor para exibir registos. Serviços auxiliares, como ImageManager, SoundManager, FileChooserService e AlertManager, são invocados pela interface sempre que necessário.

FACADE (CHESSGAMEMANAGER)

ChessGameManager centraliza toda a lógica pública que a interface pode necessitar. Mantém um ChessGame com o estado corrente, além de um CareTaker que guarda instantâneos do jogo para operações de desfazer e refazer. Também dispara eventos de propriedade para que a interface atualize o tabuleiro, o texto de estado e o histórico de log.

CAMADA DE MODELO

ChessGame contém um GameBoard e informações de jogadores, turno, resultado e estado da partida. GameBoard administra uma lista de objetos Piece. A hierarquia de Piece tem uma super-classe com atributos comuns e seis sub-classes especializadas (King, Queen, Rook, Bishop, Knight, Pawn) que conhecem os seus movimentos específicos. Position e PositionData guardam coordenadas.

Player e PlayerData representam utilizadores. Os enumerados EPieceType, EPawnMoved, ETeamColor e EChessState identificam tipos de peça, progresso de peões, equipa e situação da partida.

PieceFactory cria objetos a partir de parâmetros, enquanto PieceFactoryText cria a partir de texto para importação. ModelLog recebe entradas para fins de depuração.

PADRÕES EVIDENCIADOS

MVC está refletido na separação entre Interface, Facade e Modelo. Factory elimina dependências concretas na criação de peças. Memento aparece na dupla CareTaker e ChessGame (IOriginator) que regista e repõe estados sem expor detalhes internos.

7 Funcionalidades e Estado de Implementação

Funcionalidade Esperada	Status	Observações
Definição do Modelo de Dados (AulaP07)	Implementada totalmente	As classes de dados (Piece, Pawn, Rook, Knight, Bishop, King, Queen, GameBoard, Position, PositionData, Player, PlayerData) estão definidas, com construtores e atributos adequados e comentários básicos em Javadoc onde necessário.
Movimentos Básicos de Peças (AulaP07/AulaP10)	Implementada totalmente	Cada subclasse de Piece (Pawn, Rook, Knight, Bishop, King, Queen) possui método isValidMove(...) que implementa corretamente as regras de movimento básicas, e método move(...) que realiza as capturas quando apropriado.
Validação de Xeque / Xeque-mate (AulaP07)	Implementada totalmente	Métodos GameBoard.isKingInCheck(...) e GameBoard.isCheckMate(...) funcionam conforme esperado e são chamados por ChessGame para identificar xeque e checkmate antes de confirmar o movimento.
Movimento Especial “Roque” (AulaP07)	Implementada totalmente	No King.move(...), há detecção de deslocamento de 2 colunas, verificação de que nem rei nem torre foram movidos anteriormente, checagem de caminho livre e atualização das duas peças.
Movimento Especial “En Passant” (AulaP07)	Implementado parcialmente	A regra está implementada em Pawn.checkPawnMove(...), que verifica diagonal vazia, pawnStatus == ONCE, remove peão “fantasma” e faz o movimento. Funciona na maior parte dos casos, mas requer testes adicionais em cenários de undo/redo e em bordas (ex.:

		promoção logo após en passant).
Persistência de Dados e Serialização (.dat) (AulaP08)	Implementada totalmente	ChessGameSerialization implementa métodos saveGame(File) e loadGame(File) para serializar/desserializar o objeto ChessGame. Todos os atributos necessários (turno, posição das peças, histórico de undo/redo não pertinente) são corretamente gravados e restaurados.
Importação / Exportação de Estados Parciais (.txt) (AulaP08)	Implementada totalmente	Em ChessGame.importGameState(File) e ChessGame.exportGameState(File), o formato de texto "teams,piece1,piece2,..." é lido e gravado. O ChessGameManager invoca esses métodos corretamente, e há tratamento de erros para formato inválido ou arquivo inexistente.
Interface Gráfica JavaFX (Desenho do Tabuleiro e Seleção) (AulaP10)	Implementada totalmente	BoardCanvas calcula dinamicamente tamanho do tabuleiro, desenha casas com cores alternadas, rótulos "a"... "h" e "1"... "8". Ao clicar na célula, detecta seleção de peça e, em modo aprendizagem, destaca movimentos válidos usando facade.getValidMoves(...).
Validação de Entradas Inválidas (AulaP10)	Implementada parcialmente	Erros de coordenadas fora do tabuleiro geram mensagens em lastError. Contudo, não há bloqueio visual imediato na UI para cliques completamente fora de turno (por exemplo, clicar em célula sem selecionar peça ignora, mas não desabilita botão).

Observabilidade / Notificações (AulaP10)	Implementada totalmente	ChessGameManager dispara eventos PropertyChanged para propriedades como PROP_BOARD_CHANGED, PROP_TURN_CHANGED, PROP_CHECK, PROP_GAME_OVER. UI (RootPane, BoardCanvas, StatusBarPane) escuta e atualiza automaticamente, exibindo alertas de xeque e fim de jogo.
Barra de Estado e Informações de Jogador (AulaP10)	Implementada totalmente	StatusBarPane exibe nome e equipa do jogador atual a cada evento de turno; PlayerInfoPane mostra nome, equipa e pontuação estática. Ao alternar turno, a UI atualiza automaticamente.
Undo / Redo de Jogadas (AulaP08/AulaP10)	Implementada totalmente	Implementado via padrão Memento: CareTaker.save() antes de cada movimento, undo() e redo() restauram clones de GameBoard (que preservam pawnStatus e wasMoved). MenuUI associa “Undo Move” e “Redo Move” a esses comandos do ChessGameManager.
Modo de Aprendizagem (Learning Mode) (AulaP10)	Implementada totalmente	Ao ativar “Learning” no menu, BoardCanvas passa a mostrar, em cada peça selecionada, as casas válidas em dourado, consultando facade.getValidMoves(...).?
Reprodução de Sons para Movimentos (AulaP10)	Implementada totalmente	Cada movimento dispara BoardCanvas.playMoveAudioSequence(...), que monta sequência de ficheiros MP3 (captura, cheque, tipo de peça, colunas/linhas em áudio). SoundManager gere fila de reprodução.
Modo Edição de Tabuleiro (AulaP11)	Implementado totalmente	MenuUI inclui opção “Modo Edição”; ao ativar, se houver partida em progresso, pergunta para salvar, chama

		facade.startEditMode(), permitindo que, ao clicar numa célula, o usuário adicione/remova peças livremente.
Persistência das Edições em Arquivo (AulaP11)	Implementado totalmente	Ao finalizar edição com “Salvar Edição”, solicita nomes e turno inicial via AlertManager, chama facade.endEditModeWithSave(..) e, em seguida, usa ChessGameSerialization para gravar estado como partida pronta para novo jogo.
Exibir Logs em Janela Separada (AulaP12)	Implementado totalmente	LogWindow implementa PropertyChangeListener em ModelLog; cada chamada de ModelLog.addLog(...) gera nova entrada em LogWindow.TextArea ou ListView, exibindo histórico em tempo real, sem intervenção adicional do usuário.
Múltiplas Janelas (AulaP12)	Implementado totalmente	É possível abrir ao mesmo tempo a janela principal (tabuleiro) e a janela de logs, sem colisões de eventos. Ambas escutam mudanças do modelo.
Otimização de redraw ao Redimensionar (AulaP12)	Implementado totalmente	BoardCanvas escuta widthProperty() e heightProperty(), chamando draw() sempre que há alteração na dimensão. O tabuleiro é sempre redesenhado corretamente, mantendo proporção e rótulos.
Notificações de Xeque / Xeque-mate em Pop-ups (AulaP12)	Implementado totalmente	Em ChessGameManager, ao detectar xeque ou checkmate, dispara PROP_CHECK ou PROP_GAME_OVER. RootPane escuta esses eventos e exibe Alert correspondente, bloqueando a jogada até confirmação pelo utilizador.

Comentários de Movimento nos Logs (AulaP12)	Implementado totalmente	O histórico exibido em LogWindow inclui ações do modelo (“movimento inválido”, “peão promovido”, “roque executado”).
Exportar Logs de Partida em .txt (AulaP13)	Implementado totalmente	Os logs em memória podem ser exibidos e limpos em LogWindow, e existe um método dedicado que guarda esse histórico num ficheiro .txt.

8 Conclusão

O desenvolvimento deste jogo de xadrez em Java mostrou a aplicação prática dos conceitos avançados trabalhados ao longo do semestre, seguindo fielmente o padrão MVC para manter clara separação entre lógica de jogo, interface gráfica e controlo de fluxo. A camada de modelo encapsula todas as regras, desde movimentos básicos até roque, en passant e promoção de peão, assegurando que cada peça e o tabuleiro operam de acordo com as regras oficiais do xadrez. O controlador ChessGameManager intermedeia todas as interações, validando jogadas, alternando turnos, gerindo undo/redo por meio do padrão Memento e notificando a interface para redesenhar o tabuleiro ou exibir mensagens de cheque ou fim de jogo.

Na interface, construímos uma experiência responsiva e intuitiva em JavaFX, proporcionando um ecrã de boas-vindas que conduz o utilizador à seleção de novas partidas ou abertura de jogos salvos. O tabuleiro, desenhado em BoardCanvas, ajusta-se automaticamente às mudanças de dimensão da janela e destaca movimentos legais quando o modo de aprendizagem está ativo. Os menus integrados permitem aceder a todas as funcionalidades, incluindo modos de jogo, importação e exportação, além de opções de edição livre do tabuleiro. A gestão de recursos multimédia, centralizada em ImageManager e SoundManager, garante que as imagens das peças e os efeitos sonoros sejam carregados apenas uma vez e disponibilizados para uso imediato, melhorando o desempenho e a consistência visual e auditiva do programa.

Foram implementadas todas as funcionalidades essenciais previstas nas aulas P07 a P13, tais como persistência em ficheiros binários e de texto, janela de logs para visualização de eventos, modo de edição de posições, aplicação do padrão Factory para criação de peças e observabilidade por meio de PropertyChangeListeners. Alguns pontos, como atalhos de teclado, cronómetro para cada jogador e exportação direta de logs para ficheiro de texto, permanecem abertos para desenvolvimento futuro, mas não comprometem o funcionamento principal do jogo. A arquitetura modular adotada facilita a adição dessas melhorias sem necessidade de grandes mudanças na estrutura existente.

Em suma, a solução final é um jogo de xadrez completo, escalável e de fácil manutenção, que atende aos requisitos do enunciado e dos conteúdos das aulas. A aplicação demonstra não só a compreensão dos padrões de projeto, mas também a capacidade de integrar componentes multimédia e criar uma interface rica em JavaFX, resultando num produto final robusto e pronto para evoluir de acordo com necessidades futuras.

9 Referências

1. P – Aula 06 – Slides de apoio
2. P – Aula 06 – Projeto base
3. P – Aula 07 – Slides de apoio
4. P – Aula 08 – Slides de apoio
5. P – Aula 09 – Slides de apoio
6. T – JavaFX – Projeto exemplo MVC@PA
7. T – JavaFX – Projeto com MenuBar, Toolbar, Canvas
8. P – Aula 10 – Slides de apoio
9. P – Aula 10 – Exemplo construído
10. P – Aula 10 – Gravação da sessão
11. T – JavaFX – Projeto com PropertyChangeSupport e múltiplos Stage
12. P – Aula 11 – Slides de apoio
13. T – JavaFX – Projeto sobre Resources: CSS, Fontes, Imagens e Sons
14. P – Aula 12 – Slides de apoio
15. T – Exemplo da ShoppingList
16. T – Exemplo sobre Modal Dialogs e Javadoc
17. P – Aula 13 – Slides de apoio
18. T – Slides sobre Padrões
19. T – Slides sobre JavaFX
20. T – Exemplo sobre FSM + JavaFX (GumballMachineFX)

